

Day5_DataCleaning

March 18, 2026

0.0.1 Data Cleaning and Data Wrangling

What is Data Cleaning?

Data cleaning is the process of identifying and fixing (or removing) errors, inconsistencies, and inaccuracies in a dataset so it becomes reliable for analysis.

Turning “messy, real-world data” into “accurate, usable data”

****Common issues you clean:**** - Missing values (e.g., empty cells) - Duplicate records - Incorrect formats (e.g., date stored as text) - Outliers or impossible values (e.g., age = 200) - Inconsistent naming (e.g., “USA”, “U.S.”, “United States”)

What is Data Wrangling?

Data wrangling (also called data munging) is the process of transforming and structuring raw data into a format suitable for analysis or modeling.

Turning “clean data” into “analysis-ready data”

****Typical tasks:**** - Filtering rows and columns - Merging datasets - Aggregating data (e.g., totals, averages) - Creating new features (feature engineering) - Reshaping data (pivoting, melting)

Relationship Between Data Cleaning and Data Wrangling

Think of it as a pipeline:

Raw Data → Cleaning → Wrangling → Analysis / Machine Learning

- Cleaning focuses on fixing problems in the data
- Wrangling focuses on shaping the data for use

Example

Imagine a sales dataset:

Name	Country	Sales	Date
Ram	nepal	100	01-02-23
RAM	Nepal	NULL	2023/02/01

Name	Country	Sales	Date
Sita	NPL	200	Feb 1

Data Cleaning Steps: - Standardize “Ram” and “RAM” - Fix country names (“nepal”, “NPL” → “Nepal”) - Handle missing sales values - Convert dates into a consistent format

Data Wrangling Steps: - Group total sales by country - Extract month and year from the date column - Create a new column such as “Sales Category”

Why It Matters

Bad data leads to incorrect insights and poor decisions.

Even advanced machine learning models fail if the data is not clean.

“Garbage in, garbage out”

Important Functions and Operations in Data Cleaning and Data Wrangling Data Cleaning:

1. Handling Missing Values
 - `dropna()` – Remove missing values
 - `fillna()` – Fill missing values
2. Removing Duplicates
 - `drop_duplicates()` – Remove duplicate rows
3. Data Type Conversion
 - `astype()` – Change data types
 - `to_datetime()` – Convert to date format
4. Standardizing Text Data
 - `str.lower()` – Convert to lowercase
 - `str.upper()` – Convert to uppercase
 - `str.strip()` – Remove extra spaces
5. Replacing Incorrect Values
 - `replace()` – Replace wrong or inconsistent values
6. Handling Outliers
 - Filtering with conditions
 - IQR / Z-score methods
7. Renaming Columns
 - `rename()` – Standardize column names
8. Removing Irrelevant Data
 - `drop()` – Remove unnecessary columns/rows
9. Fixing Inconsistent Formats

- Date formatting
 - Numeric formatting (e.g., commas, symbols)
10. Validating Data

- Checking ranges (e.g., age > 0)
 - Ensuring data consistency
-

Data Wrangling: 1. Filtering Data

- Boolean indexing (e.g., `df[df['Sales'] > 100]`)

2. Sorting Data
 - `sort_values()` – Sort dataset
3. Grouping Data
 - `groupby()` – Group similar data
4. Aggregation
 - `agg()` – Apply functions like sum, mean, count
5. Merging Datasets
 - `merge()` – Combine datasets
6. Joining Data
 - `join()` – Join on index
7. Reshaping Data
 - `pivot()` – Wide format
 - `melt()` – Long format
8. Creating New Columns
 - Arithmetic operations
 - `apply()` – Custom transformations
9. Indexing and Selection
 - `loc[]`, `iloc[]` – Access specific data
10. Encoding / Mapping Data

- `map()` – Map values
 - `get_dummies()` – Convert categorical data
-

- Data Cleaning ensures data quality and correctness
- Data Wrangling prepares structured data for analysis

```
[1]: import pandas as pd

data = {
```

```

    "Name": ["Ram", "RAM", "Sita", "Gita", None],
    "Country": ["nepal", "Nepal", "NPL", "india", "India"],
    "Sales": [100, None, 200, 150, 300],
    "Date": ["01-02-23", "2023/02/01", "Feb 1", "2023-03-01", None]
}

df = pd.DataFrame(data)

```

```

[2]: # Cleaning
      #1. Handling Missing Values
      df.dropna()
      # Removes rows where any value is missing

      df["Sales"].fillna(df["Sales"].mean())
      # Replaces missing Sales values with the average Sales

```

```

[2]: 0    100.0
      1    187.5
      2    200.0
      3    150.0
      4    300.0
      Name: Sales, dtype: float64

```

```

[3]: #2. Removing Duplicates
      df.drop_duplicates(inplace=True)
      # Removes duplicate rows from the dataset

```

```

[4]: # 3. Data Type Conversion
      df["Sales"] = df["Sales"].astype(float)
      # Converts Sales column to float type

      df["Date"] = pd.to_datetime(df["Date"], format="mixed")
      # Converts Date column to datetime format, invalid values become NaT

```

```

[5]: #4. Standardizing Text Data

      df["Sales"] = df["Sales"].astype(float)
      # Converts Sales column to float type

      df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
      # Converts Date column to datetime format, invalid values become NaT

```

```

[6]: #5. Replacing Incorrect Values
      df["Country"] = df["Country"].replace({"nepal": "Nepal", "NPL": "Nepal"})

```

```

[7]: #6. Handling Outliers\
      df = df[df["Sales"] < 1000]

```

```
[8]: # 7. Renaming Columns
df.rename(columns={"Sales": "Total_Sales"}, inplace=True)
# Renames column for better readability
```

```
[9]: #8. Removing Irrelevant Data
df.drop(columns=["Name"], inplace=True)
# Removes unnecessary column
```

```
[10]: #9. Fixing Inconsistent Formats
df["Date"] = pd.to_datetime(df["Date"])
# Ensures all date values follow the same format
```

```
[11]: #10. Validating Data
df = df[df["Total_Sales"] > 0]
# Keeps only valid sales values greater than 0
```

```
[12]: #Data Wrangling
#1. Filtering Data
df[df["Total_Sales"] > 150]
# Selects rows where sales are greater than 150
```

```
[12]:   Country  Total_Sales   Date
2   Nepal         200.0 1-02-01
4   India         300.0    NaT
```

```
[13]: # 2. Sorting Data
df.sort_values(by="Total_Sales", ascending=False)
# Sorts data from highest to lowest sales
```

```
[13]:   Country  Total_Sales   Date
4   India         300.0    NaT
2   Nepal         200.0 1-02-01
3   india         150.0 2023-03-01
0   Nepal         100.0 2023-01-02
```

```
[14]: # 3. Grouping Data
df.groupby("Country")
# Groups data based on country
```

```
[14]: <pandas.api.typing.DataFrameGroupBy object at 0x7defaf8a7a40>
```

```
[15]: # Aggregation
df.groupby("Country")["Total_Sales"].sum()
# Calculates total sales per country
```

```
[15]: Country
India    300.0
Nepal    300.0
```

```
india    150.0
Name: Total_Sales, dtype: float64
```

```
[16]: # 5. Merging Datasets
df2 = pd.DataFrame({"Country": ["Nepal", "India"], "Region": ["Asia", "Asia"]})

pd.merge(df, df2, on="Country")
# Combines df with df2 based on Country column
```

```
[16]:   Country  Total_Sales   Date Region
0   Nepal         100.0 2023-01-02   Asia
1   Nepal         200.0   1-02-01   Asia
2   India         300.0         NaT   Asia
```

```
[17]: #6 Joining Data
df.join(df2.set_index("Country"), on="Country")
# Joins another dataset using index alignment
```

```
[17]:   Country  Total_Sales   Date Region
0   Nepal         100.0 2023-01-02   Asia
2   Nepal         200.0   1-02-01   Asia
3   india         150.0 2023-03-01    NaN
4   India         300.0         NaT   Asia
```

```
[18]: # Creating New Columns
df["Sales_Category"] = df["Total_Sales"].apply(
    lambda x: "High" if x > 200 else "Low"
)
# Creates a new column categorizing sales
```

```
[19]: # 9. Indexing and Selection
df.loc[0]
# Access row by label

df.iloc[0]
# Access row by position
```

```
[19]: Country                Nepal
Total_Sales              100.0
Date          2023-01-02 00:00:00
Sales_Category              Low
Name: 0, dtype: object
```

```
[20]: #10. Encoding / Mapping Data
df["Country_Code"] = df["Country"].map({"Nepal": 1, "India": 2})
# Converts categorical country values into numeric codes

pd.get_dummies(df["Country"])
```

```
# Creates one-hot encoded columns for countries
```

```
[20]:   India  Nepal  india
      0  False   True  False
      2  False   True  False
      3  False  False   True
      4   True  False  False
```

0.0.2 Date Preprocessing

Date preprocessing is a fundamental step in data cleaning and wrangling. Real-world data often contains messy, inconsistent, or incomplete date values, so we must standardize and transform them before analysis.

Date preprocessing means: - Converting raw date data into a proper datetime format - Fixing inconsistencies - Extracting useful information (year, month, etc.) - Preparing data for analysis

Example raw data:

```
“python data = [“01-02-23”, “2023/02/01”, “Feb 1”, “2023-03-01”, None]
```

```
[27]: import pandas as pd

data = ["01-02-23", "2023/02/01", "Feb 1", "2023-03-01", None]

df = pd.DataFrame({"Date": data})
df
```

```
[27]:      Date
0    01-02-23
1  2023/02/01
2      Feb 1
3  2023-03-01
4         NaN
```

```
[29]: df["Date"] = pd.to_datetime(df["Date"])
df
# Converts values to datetime
# Invalid values become NaT
```

```
[29]:      Date
0 2023-01-02
1 2023-02-01
2   1-02-01
3 2023-03-01
4         NaT
```

```
[31]: df["Date"] = df["Date"].replace({
        "Feb 1": "2023-02-01"
    })
df
```

```
[31]:      Date
0 2023-01-02
1 2023-02-01
2 2023-02-01
3 2023-03-01
4      NaT
```

```
[32]: # Convert with proper settings
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True) # DD-MM-YY format
# for eg: feb,1-> 01-02-2024

print(df)
```

```
      Date
0 2023-01-02
1 2023-02-01
2 2023-02-01
3 2023-03-01
4      NaT
```

```
[36]: # Convert date from one format to another
# Example:
# "2021/02/01" -> "01-02-2021"

# step 1: Convert to Datetime
df["Date"] = pd.to_datetime(df["Date"])

# Step 2: convert to desired format using strftime
df["Formatted_Date"] = df["Date"].dt.strftime("%d-%m-%Y")
# Convert datetime -> string in new format
df
```

```
[36]:      Date Formatted_Date
0 2023-01-02      02-01-2023
1 2023-02-01      01-02-2023
2 2023-02-01      01-02-2023
3 2023-03-01      01-03-2023
4      NaT           NaN
```

```
[37]: df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Day"] = df["Date"].dt.day
```

```
df
```

```
[37]:
```

	Date	Formatted_Date	Year	Month	Day
0	2023-01-02	02-01-2023	2023.0	1.0	2.0
1	2023-02-01	01-02-2023	2023.0	2.0	1.0
2	2023-02-01	01-02-2023	2023.0	2.0	1.0
3	2023-03-01	01-03-2023	2023.0	3.0	1.0
4	NaT	NaN	NaN	NaN	NaN

```
[39]: df["Day_Name"] = df["Date"].dt.day_name()  
# Example: Monday  
df
```

```
[39]:
```

	Date	Formatted_Date	Year	Month	Day	Day_Name
0	2023-01-02	02-01-2023	2023.0	1.0	2.0	Monday
1	2023-02-01	01-02-2023	2023.0	2.0	1.0	Wednesday
2	2023-02-01	01-02-2023	2023.0	2.0	1.0	Wednesday
3	2023-03-01	01-03-2023	2023.0	3.0	1.0	Wednesday
4	NaT	NaN	NaN	NaN	NaN	NaN

```
[ ]:
```